

第六章 回溯法



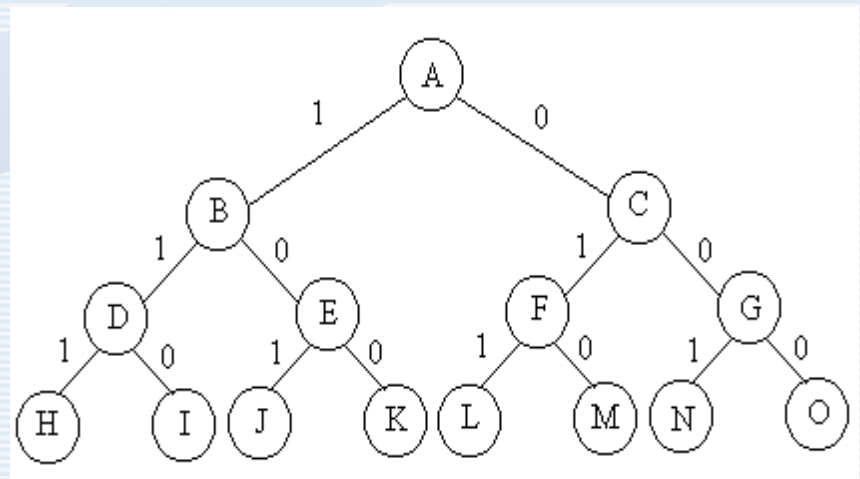
学习要点

- 理解回溯法的概念。着重讨论可以用回溯法求解的问题的一般特征。
- 掌握回溯算法的基本要素
- 理解回溯算法的一般理论
- 通过应用范例学习。
 - (1) 一般方法;
 - (2) 8皇后问题;
 - (3) 子集和数的问题; ➡
 - (4) 图的着色问题

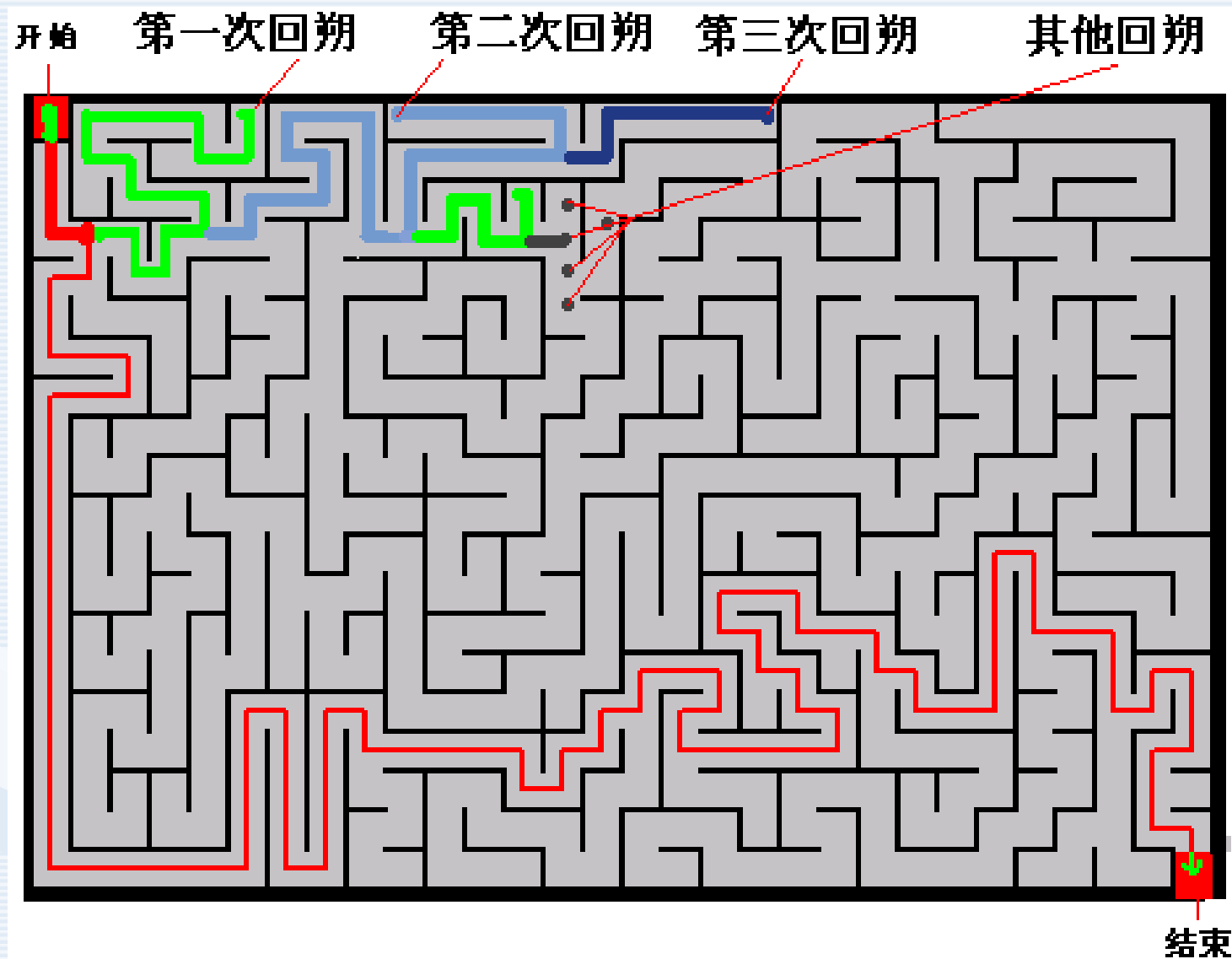
6.1 回溯法的一般方法

- 回溯法有“**通用的解题法**”之称。
- 有许多问题，找出它的解集或者要求回答什么解是满足某些约束条件的最佳解时，往往要使用回溯法。
- 回溯法的基本做法是搜索，或是一种组织得井井有条的，能避免不必要搜索的穷举式搜索法。这种方法适用于解一些组合数相当大的问题。
- **解决一个问题的所有可能的决策序列构成该问题的解空间。**解空间中满足约束条件的决策序列称为可行解。一般说来，解任何问题都有一个目标，在约束条件下使目标达到最优的可行解称为该问题的最优解。

- 回溯法的基本思想是在一棵含有问题全部可行解的状态空间树上，**从根结点出发进行深度优先搜索，解在叶子结点上。**搜索过程中，每到达一个结点时，则判断该结点为根的子树是否含有问题的解，如果可以确定该子树中不含有问题的解，则放弃对该子树的搜索，**退回到上层父结点，继续**下一步深度优先搜索过程。
- 在回溯法中，**并不是先构造出整棵状态空间树，再进行搜索，而是在搜索过程，逐步构造出状态空间树，即边搜索，边构造。**



- 例：迷宫游戏



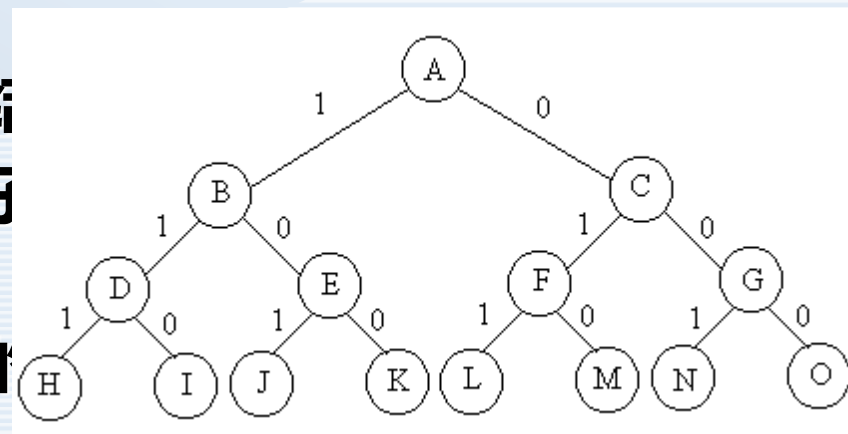
有关问题状态的几个术语

- **问题状态**：树中的每一个结点确定所求解问题的一个问题状态。代表问题状态的结点称为问题结点
- **状态空间**：由根结点到其它结点的所有路径确定了这个问题的状态空间。
- **解状态**：是这样的一些问题状态S，对于这些问题状态，由根到S的那条路径确定了解空间的一个元组。其对应的结点称为解结点。
- **答案状态**：是这样的一些问题状态S，对于这些问题状态，由根到S的那条路径确定了这问题的一个解（即它满足隐式约束条件）。
- **扩展结点**：一个正在产生儿子的结点称为扩展结点(E)
- **活结点**：一个自身已生成但其儿子还没有全部生成的结点称做活结点
- **死结点**：一个所有儿子已经产生的结点称做死结点

有关问题状态的几个术语

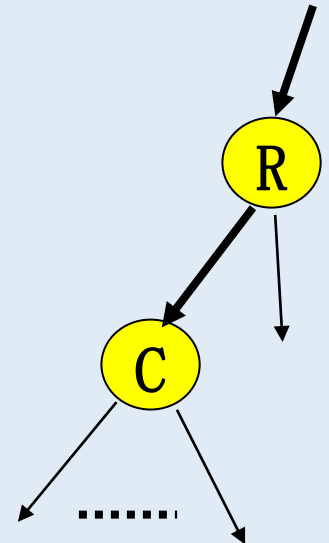
- **问题状态**：树中的每一个结点确定所求解问题的一个问题状态。代表问题状态的结点称为问题结点
- **状态空间**：由根结点到其它结点的所有路径确定了这个问题的状态空间。
- **解状态**：是这样的一个问题状态，对于这个问题状态，由根到S的结点称为解状态，**状态空间树的每个问题结点对应一个子解空间，同时也代表已经作出的一些选择。状态空间树是在算法执行中产生的。**
- **答案状态**：由根到S的结点满足问题约束条件（即隐式约束条件）。

- **扩展结点**：一个正在产生儿子的结点
- **活结点**：一个自身已生成但其儿子尚未生成，称做活结点
- **死结点**：一个所有儿子已经产生的结点



状态空间树的产生方法

- **深度优先的问题状态生成法**：如果对一个扩展结点R，一旦产生了它的一个儿子C，就把C当做新的扩展结点。在完成对子树C（以C为根的子树）的穷尽搜索之后，将R重新变成扩展结点，继续生成R的下一个儿子（如果存在）
- **回溯法**：为了避免生成那些不可能产生最佳解的问题状态，要不断地利用限界函数(bounding function)来处死那些实际上不可能产生所需解的活结点，以减少问题的计算量
具有限界函数的深度优先生成法称为回溯法



回溯法的基本步骤

- (1) 针对所给问题，定义问题的解空间；
- (2) 确定易于搜索的解空间结构；
- (3) 以深度优先方式搜索解空间，并在搜索过程中用**限界函数**避免无效搜索。

用约束函数在扩展结点处剪去不满足约束的子树；
用限界函数剪去得不到最优解的子树。

用回溯法解题的一个显著特征是在搜索过程中动态产生问题的解空间。在任何时刻，算法只保存从根结点到当前扩展结点的路径。如果解空间树中从根结点到叶结点的最长路径的长度为 $h(n)$ ，则回溯法所需的计算空间通常为 $O(h(n))$ 。而显式地存储整个解空间则需要 $O(2^{h(n)})$ 或 $O(h(n)!)$ 内存空间。

回溯法的基本思路：

- 问题的解可以用一个 n 元组 $(x_1, \dots, x_n)$ 来表示，其中的 x_i 取自于某个有穷集 S_i ，并且这些解必须使得某一限界函数 $B(x_1, \dots, x_n)$ 取极值。
- 假设集合 S_i 的大小是 m_i ，于是就有 $m = m_1 m_2 \dots m_n$ 个 n -元组可能满足函数 B 。
- 构造出这 m 个 n -元组并逐一测试它们是否满足函数 B ，从而找出该问题的所有最优解。

所以

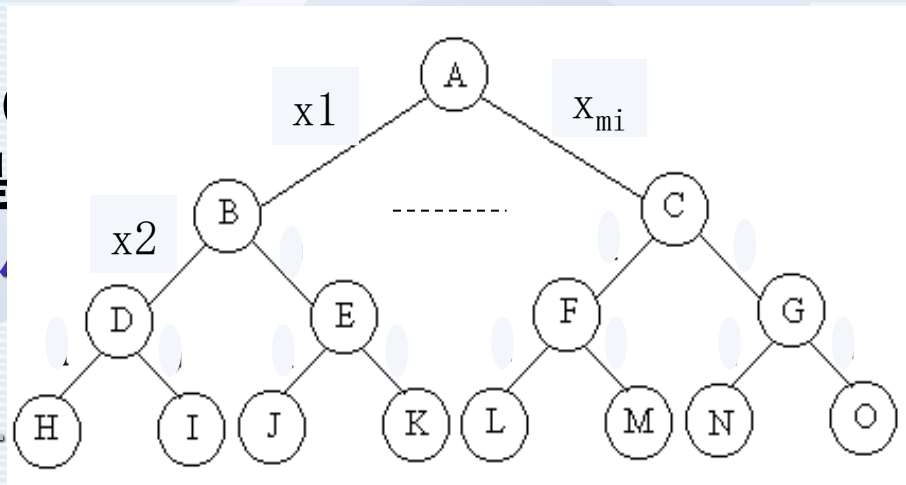
- 不断地用修改过的限界函数 $B(x_1, \dots, x_i)$ 去测试正在构造的 n 元组的部分向量 $(x_1, \dots, x_i)$ ，看是否可能导致最优解。
- 如果判定 $(x_1, \dots, x_i)$ 不能导致最优解，那么就将可能要测试的 $m_{i+1} \dots m_n$ 个向量略去。
- 因此回溯法的基本做法是搜索。或是一种组织得井井有条的，能避免不必要搜索的穷举式搜索算法。

回溯法的基本思路：

- 问题的解可以用一个 n 元组 $(x_1, \dots, x_n)$ 来表示，其中的 x_i 取自于某个有穷集 S_i ，并且这些解必须使得某一限界函数 $B(x_1, \dots, x_n)$ 取极值。
- 假设集合 S_i 的大小是 m_i ，于是就有 $m = m_1 m_2 \dots m_n$ 个 n -元组可能满足函数 B 。
- 构造出这 m 个 n -元组并逐一测试它们是否满足函数 B ，从而找出该问题的所有最优解。

所以

- 不断地用修改过的限界函数 B 对部分向量 $(x_1, \dots, x_i)$ ，看是否满足限界函数。
- 如果判定 $(x_1, \dots, x_i)$ 不能导致最优解，则略去 $m_{i+1} \dots m_n$ 个向量略去。
- 因此回溯法的基本做法是搜索。或是一棵树的穷举式搜索算法。



问题的解空间和约束条件

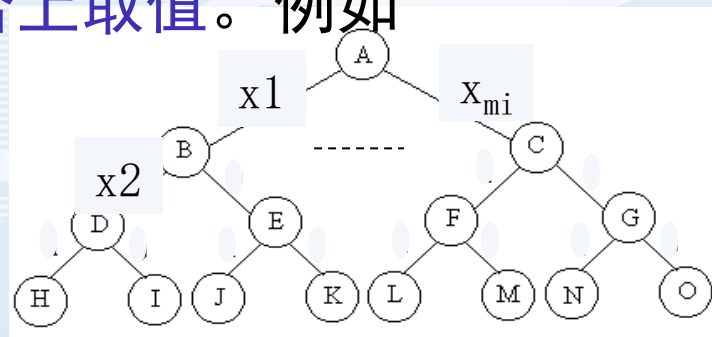
解空间：对于问题的一个实例，解向量满足显式约束条件的所有多元组，构成了该实例的一个解空间。

约束条件：用回溯法求解的许多问题都要求所有的解满足一组综合的约束条件。对 n 元组 $(x_1, x_2, \dots, x_n)$ 中的分量的约束，可分为两类：**一类是显约束，一类是隐约束**，**显约束**是限定每个 x 只从一个给定的有限集合上取值。例如

$x_i \geq 0$ $S_i = \{\text{所有非负实数}\}$

$x_i = 0$ 或 $x_i = 1$ $S_i = \{0, 1\}$

$b_i \leq x_i \leq u_i$ $S_i = \{a: b_i \leq a \leq u_i\}$



隐约束条件则规定解空间中那些实际上满足规范函数 $B_i(x_1, x_2, \dots, x_i)$ 的元组。因此，**隐约束描述了 x_i 必须彼此相关的情况**。不过，显约束和隐约束不是绝对的。有时显约束可以表示成隐约束，隐约束也可以表示成显约束。

➤例：对于有 n 种可选择物品的0/1背包问题，其解空间由长度为 n 的0-1向量组成。该解空间包含了对变量的所有可能的0-1赋值。
当 $n=3$ 时，其解空间是
 $\{(0,0,0), (0,1,0), (0,0,1), (1,0,0), (0,1,1), (1,0,1), (1,1,0), (1,1,1)\}$

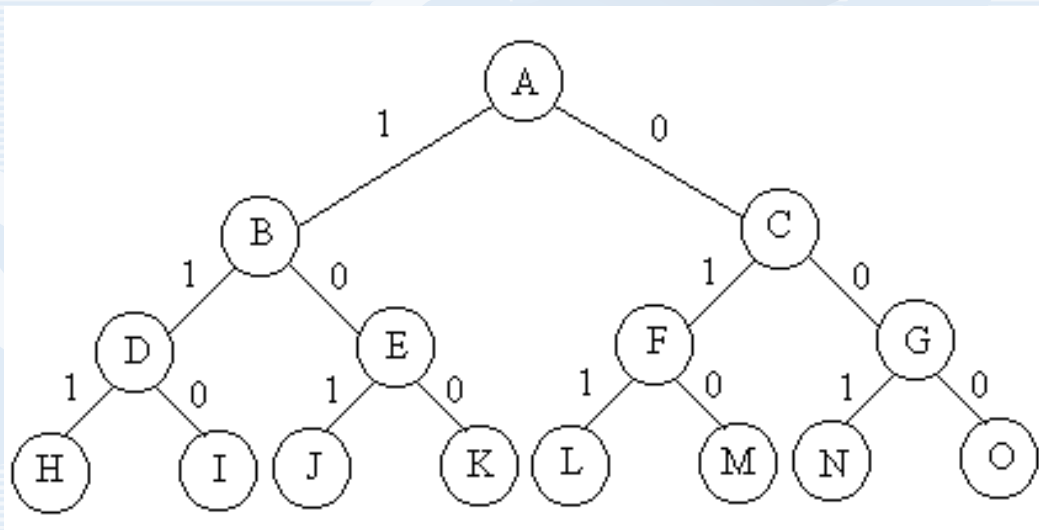
解向量： (x_1, x_2, x_3)

显式约束： 0,1

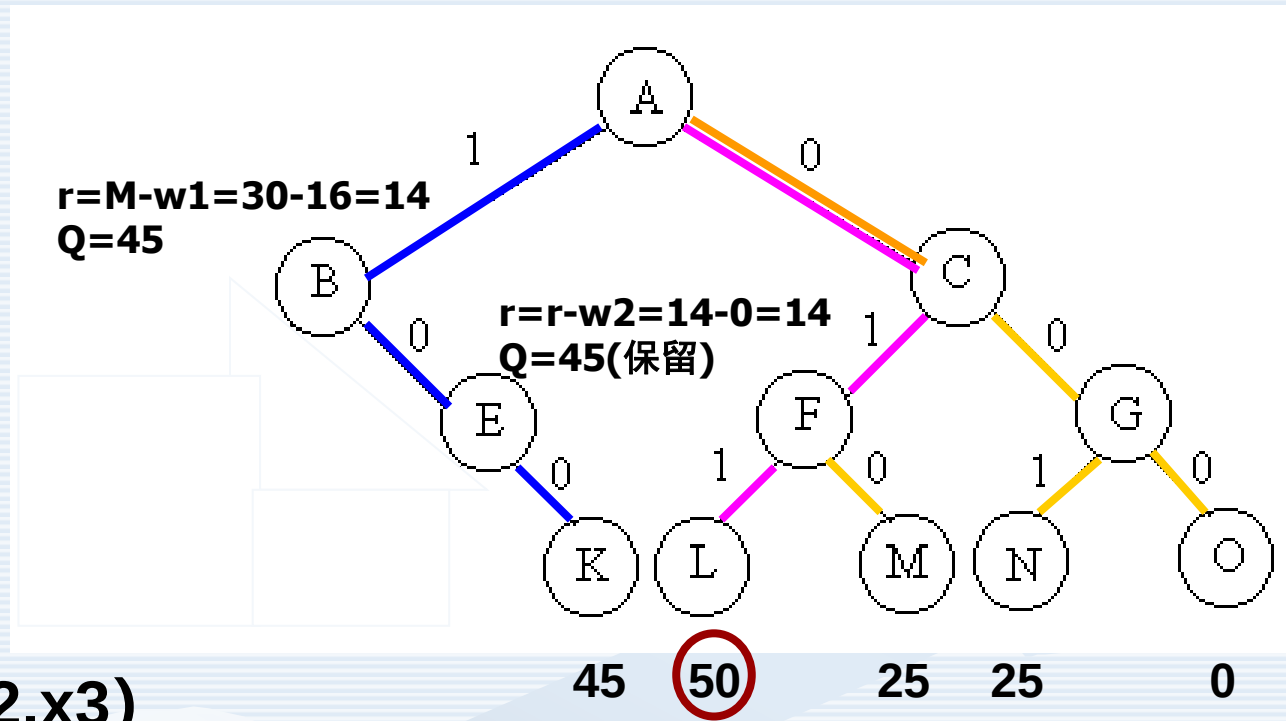
隐式约束： $x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 \leq C$

$\max(x_1 \cdot p_1 + x_2 \cdot p_2 + x_3 \cdot p_3)$

解空间： $\{(0,0,0), (0,1,0), (0,0,1), (1,0,0), (1,1,0), (1,0,1), (0,1,1), (1,1,1)\}$



如果 $n=3, M=30, w=(16,15,15), v=(45,25,25)$ ，对0/1背包问题的求解，我们要从下图的根结点开始搜索其解空间。



解向量: (x_1, x_2, x_3)

显式约束: 0,1

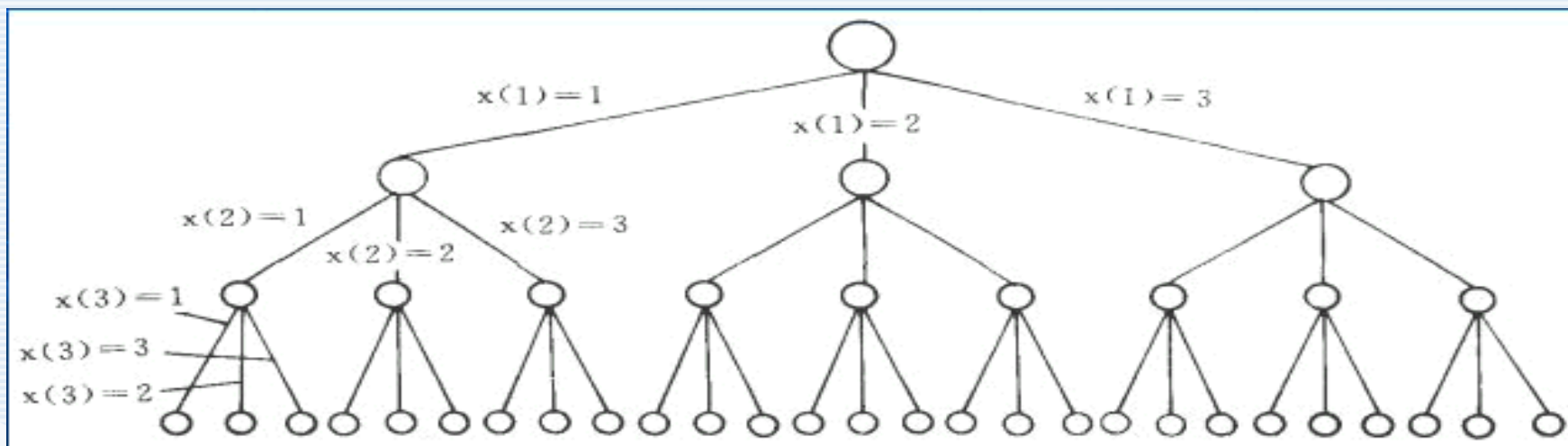
隐式约束: $x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 \leq C$

$\max(x_1 \cdot p_1 + x_2 \cdot p_2 + x_3 \cdot p_3)$

解空间: $\{(0,0,0), (0,1,0), (0,0,1), (1,0,0), (1,1,0), (1,0,1), (0,1,1), (1,1,1)\}$

深度优先搜索策略

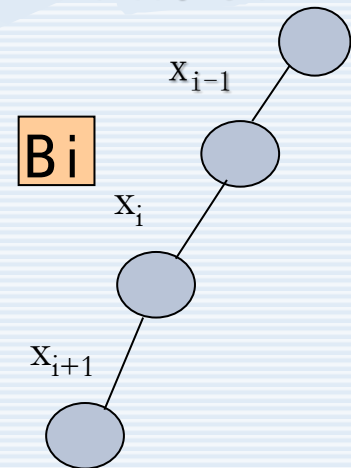
回溯法将 n 元组 $(x_1, x_2, \dots, x_n)$ 的状态空间 E 表示成一棵高度为 n 的带权有权树 T ,把在 E 中求问题 P 的所有解转化为在 T 中搜索。下图所示的是一棵高度为 $n=3$ 、每层结点数都为3的状态空间树。



从根出发进行搜索，搜索到每个结点，先判断以该结点为根的子树是否肯定不包含问题的解，如果不包含，则跳过对该子树的搜索，一层一层向祖先结点回溯，直到遇上还未搜索过的子树，才搜索这棵子树。回溯到根结点时，则找出了问题的所有解。

6.1.2 回溯算法的形式描述：

- 假设回溯法要找出所有的答案结点。
- 设 $(x_1, x_2, \dots, x_{i-1})$ 是状态空间树中由根到一个结点的路径，而 $T(x_1, \dots, x_{i-1})$ 是下述所有结点 x_i 的集合，它使得对于每一个 x_i ， $(x_1, x_2, \dots, x_i)$ 是由根到一个结点 x_i 的路径；假定还存在着一一些**限界函数** B_i ，如果路径 $(x_1, x_2, \dots, x_i)$ 不可能延伸到一个答案结点，则 $B_i(x_1, x_2, \dots, x_i)$ 取假值，否则取真值。
- 于是解向量 $X(1:n)$ 中的第 i 个分量，就是那些选自集合 $T(x_1, x_2, \dots, x_{i-1})$ 且使 B_i 为真的 x_i



6.1.2 回溯算法的形式

- 假设回溯法要找出所有的答案结

- 设 $(x_1, x_2, \dots, x_{i-1})$ 是状态空间树中

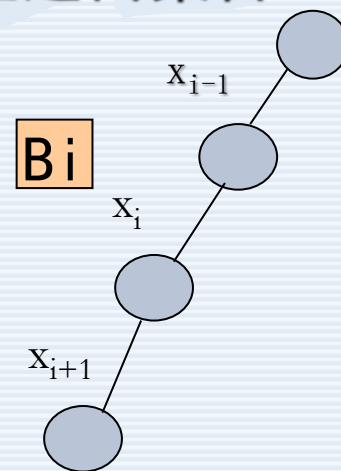
$T(x_1, \dots, x_{i-1})$ 是下述所有结点 x_i 的:

$(x_1, x_2, \dots, x_i)$ 是由根到一个结点 x_i 的路径; 假定还仔仕有一些
限界函数 B_i , 如果路径 $(x_1, x_2, \dots, x_i)$ 不可能延伸到一个答案结
 点, 则 $B_i(x_1, x_2, \dots, x_i)$ 取假值, 否则取真值。

- 于是解向量 $X(1:n)$ 中的第 i 个分量, 就是那些选自集合 T
 $(x_1, x_2, \dots, x_{i-1})$ 且使 B_i 为真的 x_i

```

Procedure search(当前状态)
if 当前状态是目标状态
    then exit;
for 对所有可能的状态 do
    search(新状态)
repeat
End
    
```



算法6.1 回溯法的一般方法

Procedure **BACKTRACK**(n)

k=1; //考察第1层

while(k>0) do //k=0时, 结点被搜索完

if 还剩有没检验的 x_k 使得

$x_k \in T_k(x_1, \dots, x_{k-1})$ and $B_k(x_1, x_2, \dots, x_k)$

// $T_k = T_k(x_1, \dots, x_{k-1})$ 中未取过的值; x_k 的取值与前k-1个有关

then if $(x_1, \dots, x_k)$ 是一答案路径; //可能成为解, 或需深度搜索

then 输出 $(x_1, \dots, x_k)$; endif //是一条已抵达一答案结点的路径

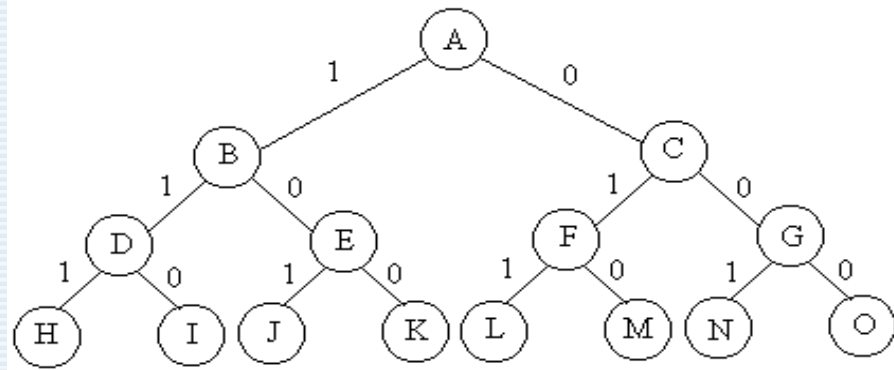
k=k+1; //深度扩展搜索, 向下一层

else k=k-1 //试探完了所有的 x_k , 回溯到上一层

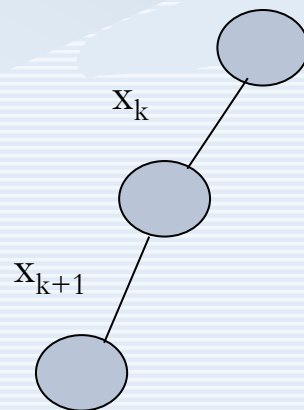
endif

repeat

End BACKTRACK



//每个 x_k 是1棵子树



算法6.1 回溯法的一般方法

Procedure **BACKTRACK**(n)

k=1; //考察第1层

while(k>0) do //k=0时, 结点被搜索完

if 还剩有没检验的 x_k 使得

$x_k \in T_k(x_1, \dots, x_{k-1})$ and $B_k(x_1, x_2, \dots, x_k)$

// $T_k = T_k(x_1, \dots, x_{k-1})$ 中未取过的值; x_k 的取值与前k-1个有关

then if $(x_1, \dots, x_k)$ 是一答案路径; //可能成为解, 或需深度搜索

then 输出 $(x_1, \dots, x_k)$; endif //是一条已抵达一答案结点的路径

k=k+1; //深度扩展搜索, 向下一层

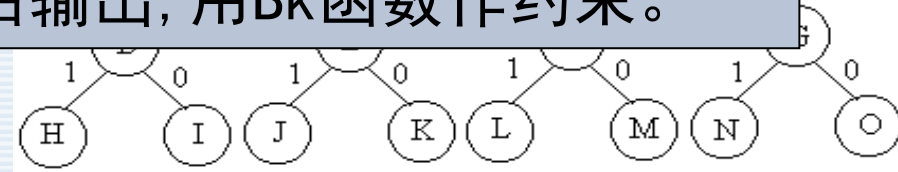
else k=k-1 //试探完了所有的 x_k , 回溯到上一层

endif

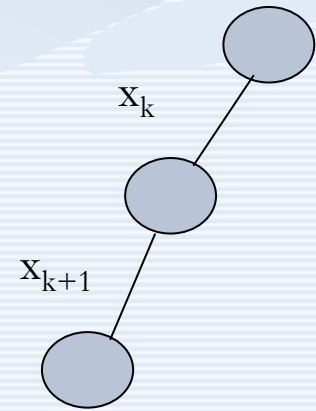
repeat

End BACKTRACK

对控制流程的抽象描述, 每个解在 $x(1:n)$ 中生成, 求出解后输出, 用 B_k 函数作约束。



//每个 x_k 是1棵子树



算法6.2 递归回溯算法

设对于任意的 $1 \leq k \leq n$ ，状态结点 $(x_1, x_2, \dots, x_{k-1})$ 被激活以后，满足显约束的 x_k 的全体解集 $T_k(x_1, x_2, \dots, x_{k-1})$ ，而满足隐约束的 x_k 当且仅当逻辑表达式 $B_k(x_1, x_2, \dots, x_k)$ 为真。

Procedure **RBACKTRACK(k)**

global $n, X(1:n)$ // n 控制深度

if $k > n$ then return; /表示搜索完，达到空间树的深度，结束

else

for $x_k \in T_k(x_1, \dots, x_{k-1})$ and $B_k(x_1, x_2, \dots, x_k)$ do

if $k == n$ then 输出 $(X_1, \dots, X_n)$ endif

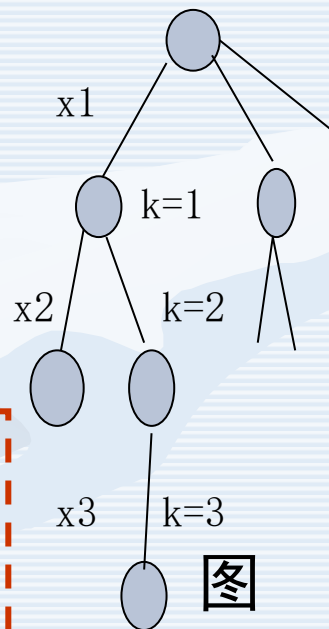
call **RBACKTRACE**($k+1$); //深度扩展搜索

repeat

endif

End **RBACKTRACK**

产生所有解，只需要一个解时，要改造。



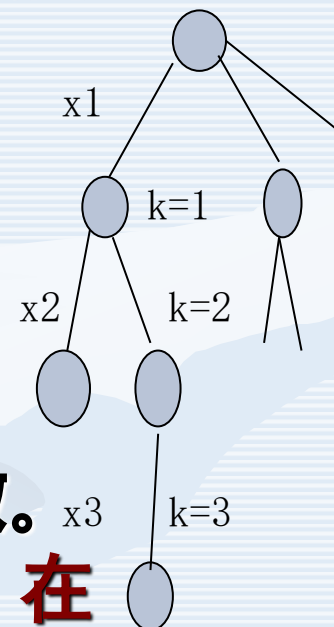
对于所有可以由根结点到达，并可能使界限函数 B 为真的结点 $X(k)$ ，判断 $X(1), \dots, X(k)$ 是否是一条已抵达一答案结点的路径

回溯法效率分析

通过前面具体实例的讨论容易看出，回溯算法的效率在很大程度上依赖于以下因素：

- (1)产生下一个 $x[k]$ 的时间；
- (2)满足显约束条件的 $x[k]$ 值的个数；
- (3)计算约束函数 B_i 的时间；
- (4)满足约束函数 B_i 的所有 $x[k]$ 的个数。

好的约束函数能显著地减少所生成的结点数。但这样的约束函数往往计算量较大。因此，在选择约束函数时通常存在生成结点数与约束函数计算量之间的折衷。



6.2 n皇后问题

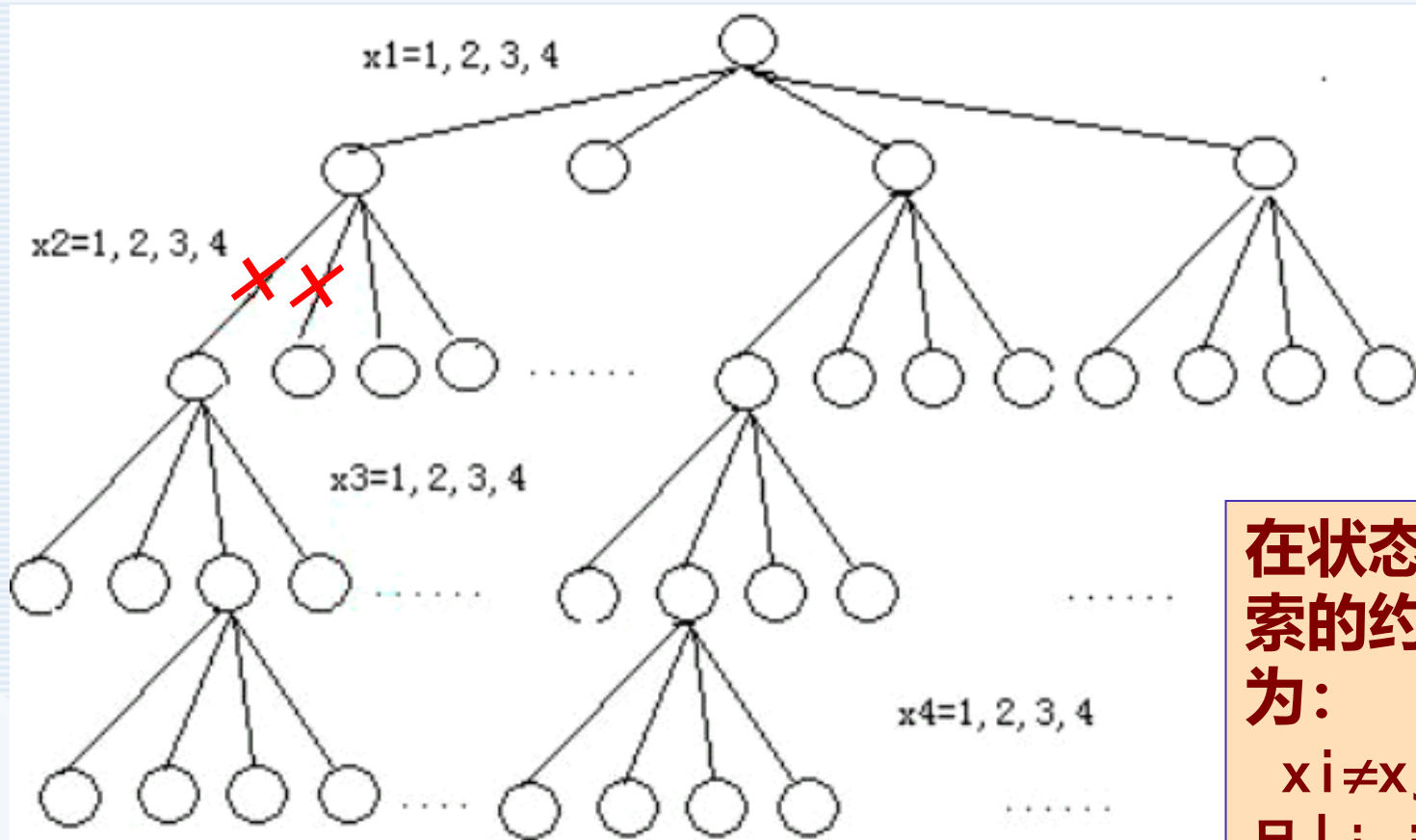
- **问题描述：**找出在一个 $n \times n$ 棋盘上放置 n 个皇后，且使得每两个之间都不能互相“攻击”，也就是使得每两个都不能在同一行、同一列及同一条斜角线上的所有方案。
- **问题的状态：**即棋盘的布局状态，状态空间树的根为空棋盘，每个布局的下一步可能布局为该布局结点的子结点；
- **n皇后问题可以表示为n-元组 $(x_1, \dots, x_n)$** ，其中 x_i 是放置皇后 i 所在的列号。

例： $n=4$ ，即4皇后问题。用4元组 $(x_1, \dots, x_4)$, $i=1, 2, 3, 4$, 表示问题的解，其中 x_i 表示把第 i 个皇后放在**第 i 行的第 x_i 列**。所以

(1) 显式约束条件： $x_i \in S_i = \{1, 2, 3, 4\}$, $1 \leq i \leq 4$

(2) 隐式约束条件：没有两个 x_i 可以相同且没有两个皇后可以在同一条斜角线上。

4-皇后问题的状态空间树

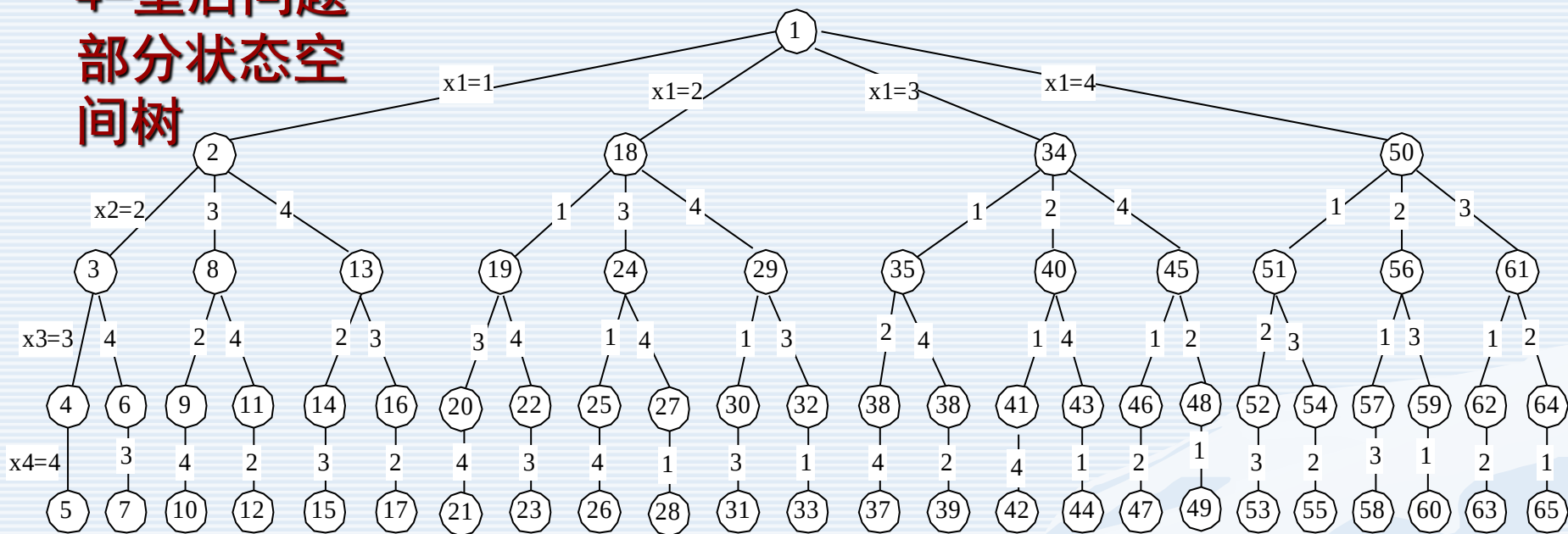


在状态空间树上搜索的约束(隐约束)为:

$x_i \neq x_j \quad i \neq j,$
且 $|j-i| \neq |x_j - x_i|$
这是皇后不同列且不在一条斜角线的约束。

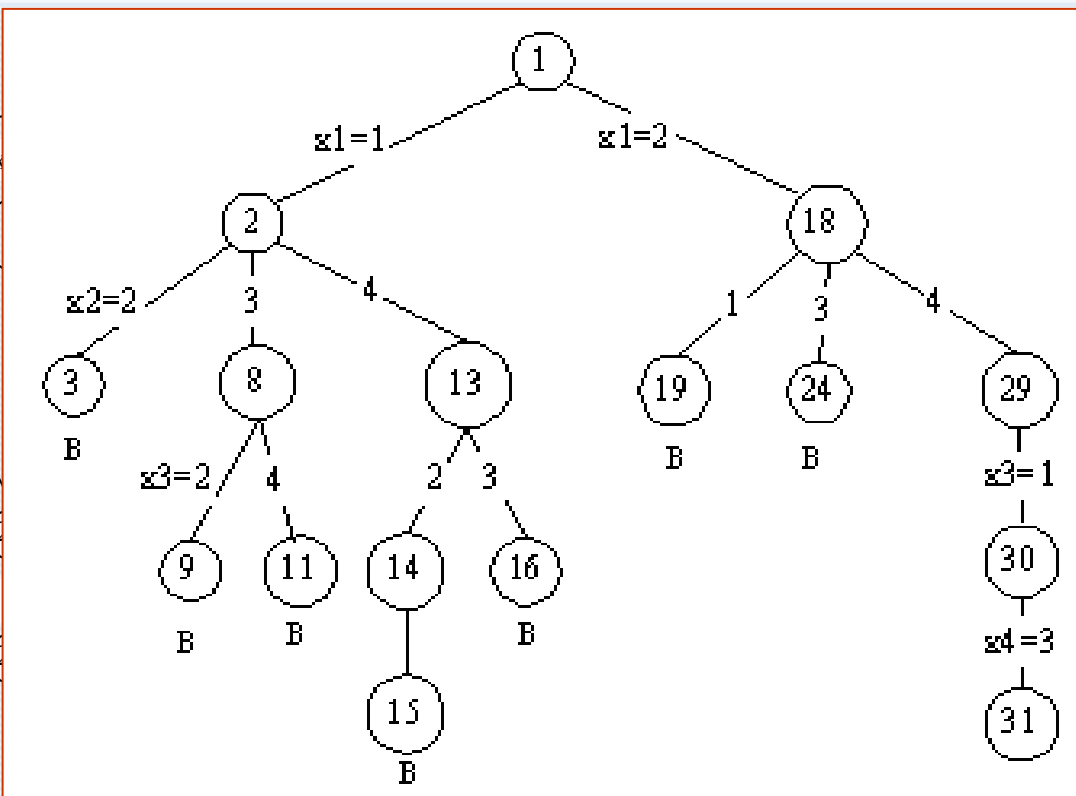
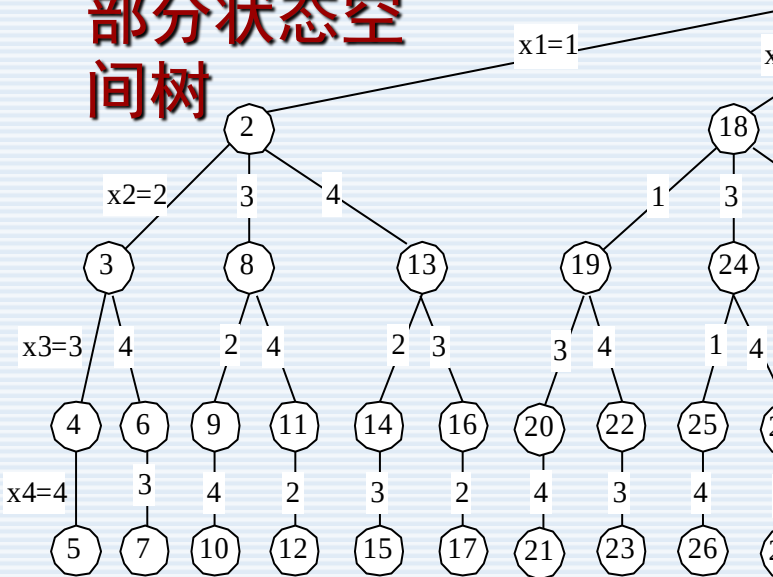
考察发现有许多结点是可以不要搜索的, 例如 $x_1=1, x_2=1$ 的子树。因此, 对于每个结点需要用判定函数进行考察, 不能使判定函数为真的结点就处死。

4-皇后问题 部分状态空 间树

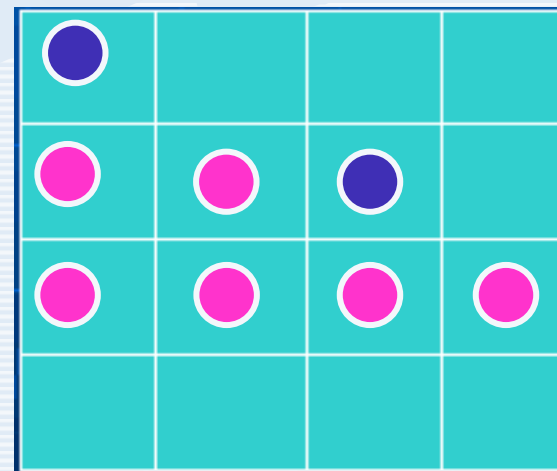


用过程PLACE(k)测试待确定的第k皇后是否和前面已确定的k-1个皇后是否在同一列、同一条斜角线，第k个皇后只要与任一皇后在同一列、同一条斜角线，就返回false,当第k个皇后能放置于 $x(k)$ 的当前位置时，这个返回值为真。

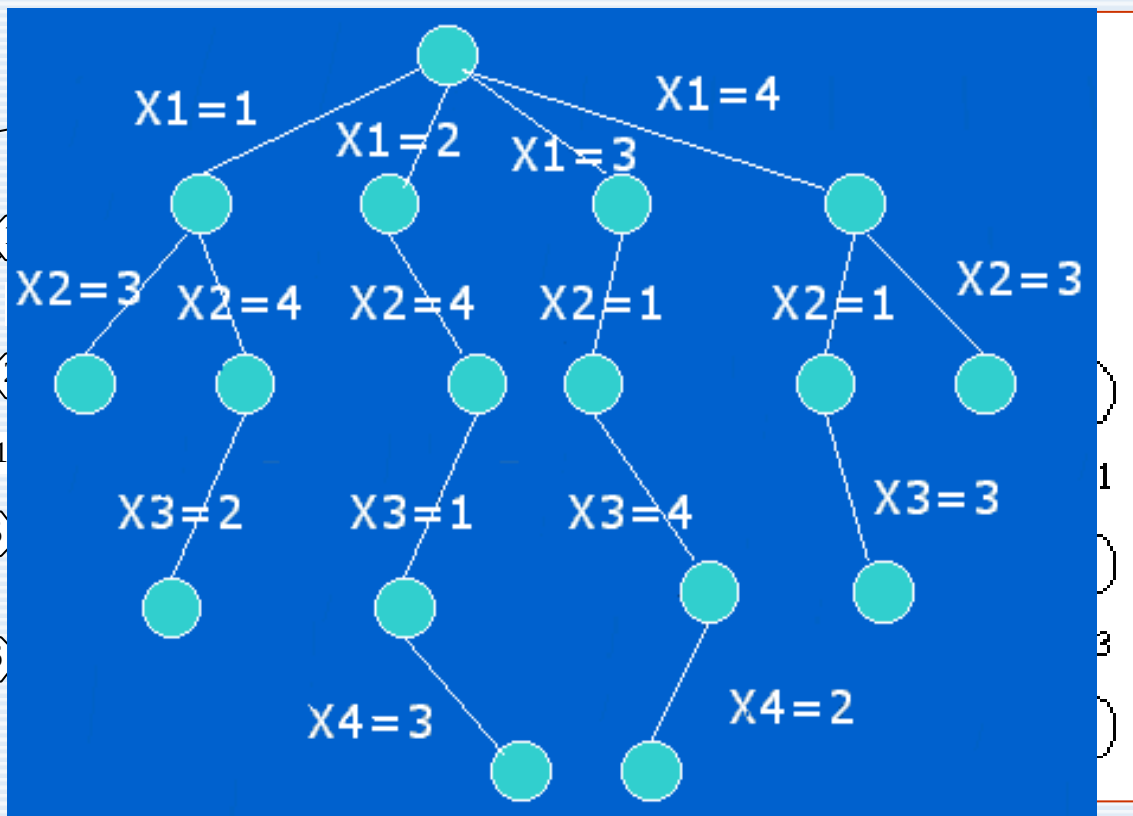
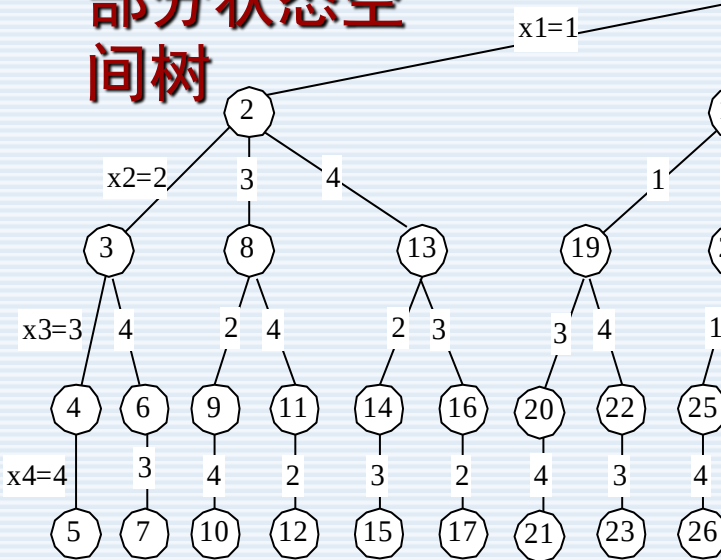
4-皇后问题 部分状态空间树



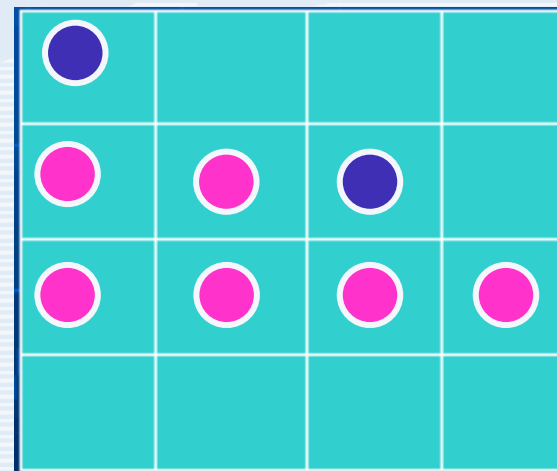
用过程PLACE(k)测试待确定的第k皇后是否和前面已确定的k-1个皇后是否在同一列、同一条斜角线，第k个皇后只要与任一皇后在同一列、同一条斜角线，就返回false,当第k个皇后能放置于x(k)的当前位置时，这个返回值为真。



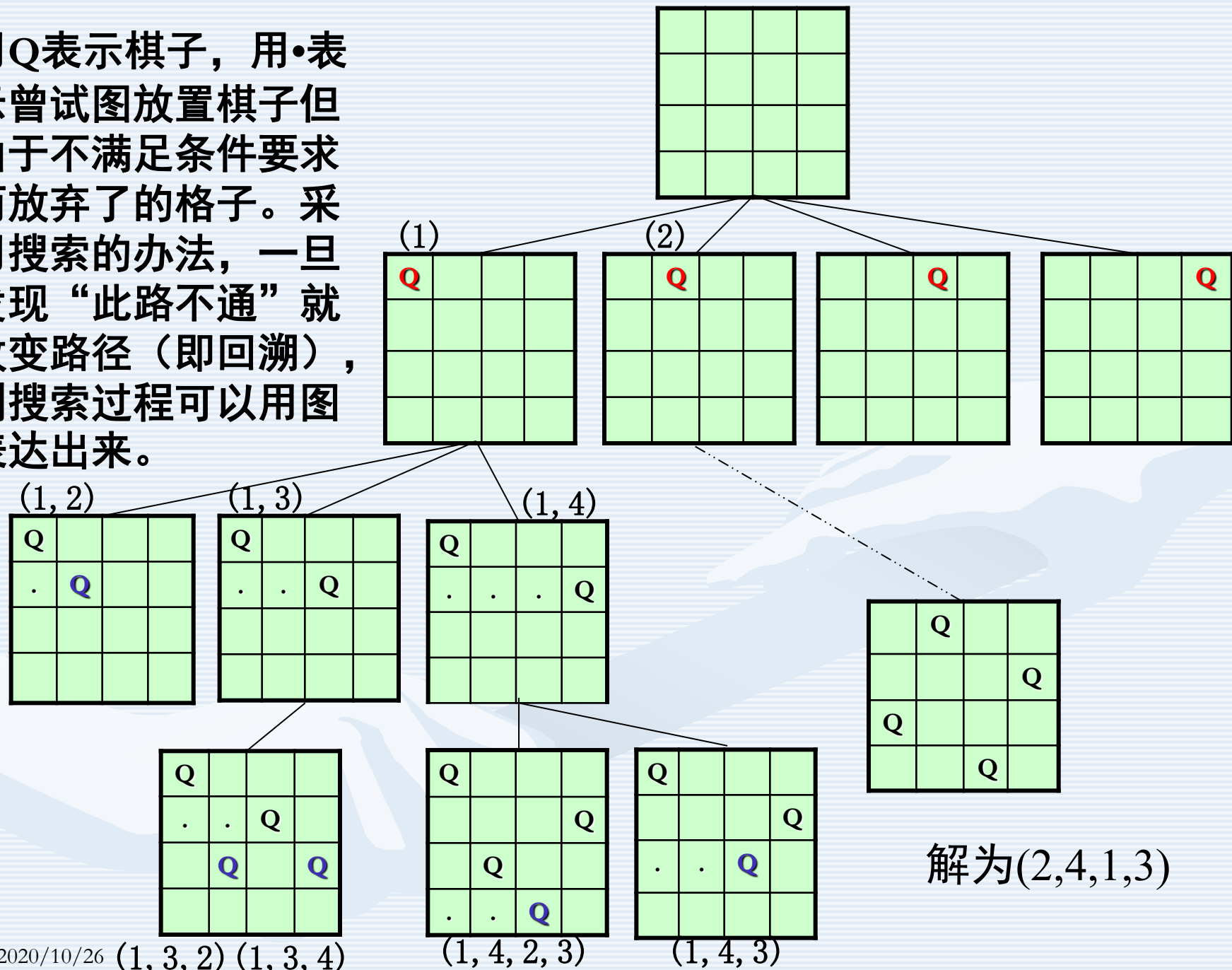
4-皇后问题 部分状态空间树



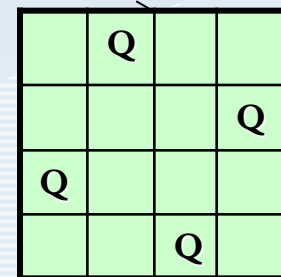
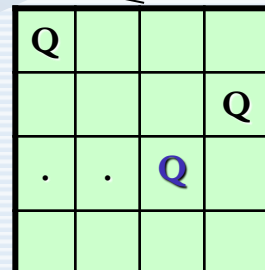
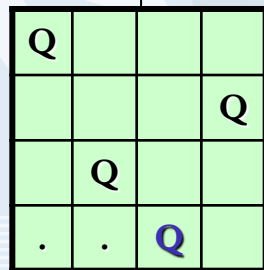
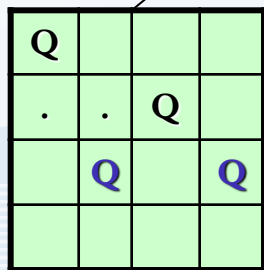
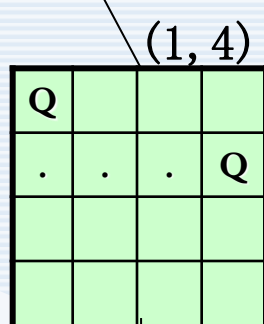
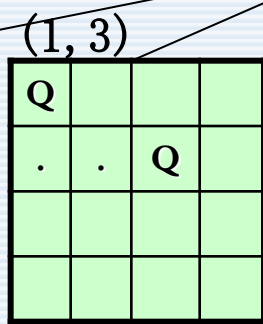
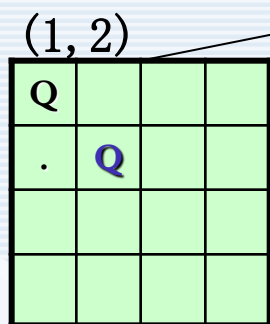
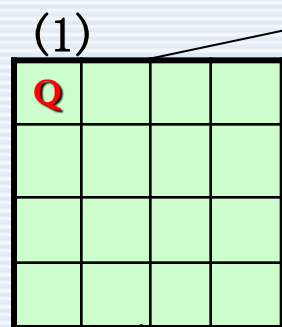
用过程PLACE(k)测试待确定的第k皇后是否和前面已确定的k-1个皇后是否在同一列、同一条斜角线，第k个皇后只要与任一皇后在同一列、同一条斜角线，就返回false,当第k个皇后能放置于 $x(k)$ 的当前位置时，这个返回值为真。



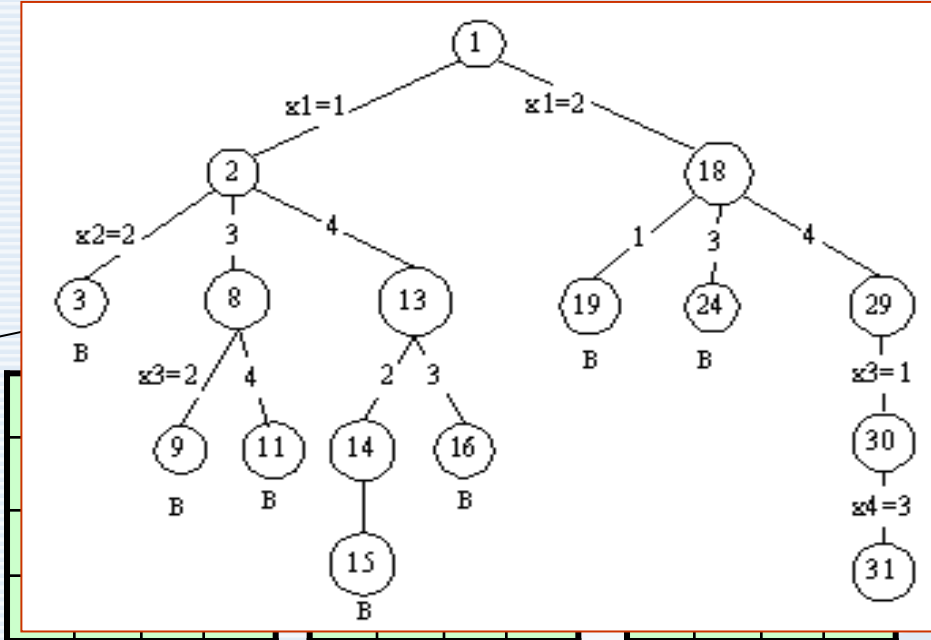
用Q表示棋子，用•表示曾试图放置棋子但由于不满足条件要求而放弃了的格子。采用搜索的办法，一旦发现“此路不通”就改变路径（即回溯），则搜索过程可以用图表达出来。



用Q表示棋子，用•表示曾试图放置棋子但由于不满足条件要求而放弃了的格子。采用搜索的办法，一旦发现“此路不通”就改变路径（即回溯），则搜索过程可以用图表达出来。



解为(2,4,1,3)



例： 8-皇后问题

- 8-皇后问题实际上很容易一般化为 n -皇后问题，即要求找出一个 $n \times n$ 棋盘上放置 n 个皇后并使其不能互相攻击的所有方案。
- 令 $(x_1, \dots, x_n)$ 表示一个解，由于没有两个皇后可以放入同一列，因此所有的 x_i 将是互不相同的，一行只能一个皇后。

不同列、不同行的表示： $x_i \neq x_j, i \neq j$

- 那么关键是应如何去测试两个皇后是否在同一条斜角线上呢？

不在斜角线上： $|j-i| \neq |x_j - x_i|$

约束条件： $x_i \neq x_j, i \neq j,$
且 $|j-i| \neq |x_j - x_i|$

1 2 3 4 5 6 7 8

1			Q				
2					Q		
3							Q
4	Q						
5						Q	
6	Q						
7			Q				
8				Q			

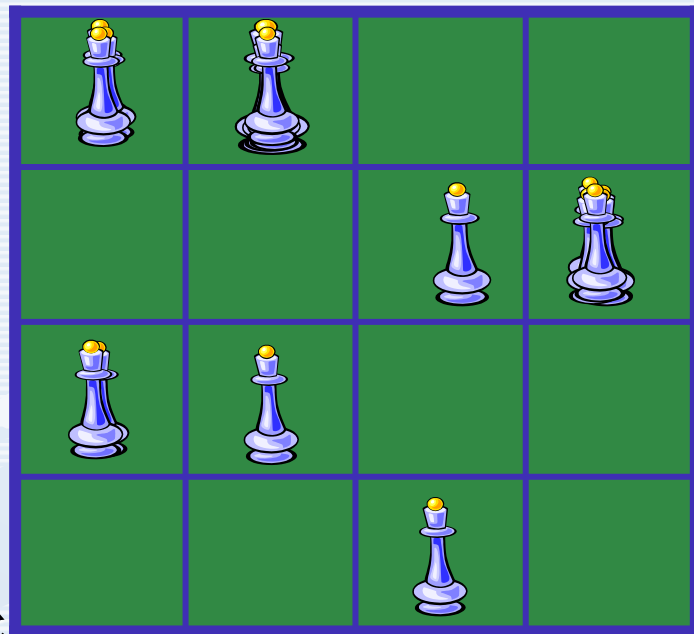
• 由于解向量之间不能相同，所以解空间的大小由 8^8 个元组减少到 $8!$ 个元组。图中的解表示为一个8-元组即 **(4, 6, 8, 2, 7, 1, 3, 5)**。

- 如果用二维数组 $A(1:n, 1:n)$ 的下标来标记每个皇后的位置，那么可以看到，**对于在同一条斜角线上的由左上方到右下方的每一个元素有相同的“行-列”值，同样，在同一条斜角线上的由右上方到左下方的每一个元素则有相同的“行+列”值。**

a₁₁	a ₁₂	a ₁₃	a ₁₄	a ₁₅	a ₁₆	a ₁₇	a ₁₈
a ₂₁	a₂₂	a ₂₃	a ₂₄	a ₂₅	a ₂₆	a ₂₇	a ₂₈
a ₃₁	a ₃₂	a₃₃	a ₃₄	a ₃₅	a ₃₆	a ₃₇	a ₃₈
a ₄₁	a ₄₂	a ₄₃	a₄₄	a ₄₅	a ₄₆	a ₄₇	a ₄₈
a ₅₁	a ₅₂	a ₅₃	a ₅₄	a₅₅	a ₅₆	a ₅₇	a ₅₈
a ₆₁	a ₆₂	a ₆₃	a ₆₄	a ₆₅	a₆₆	a ₆₇	a ₆₈
a ₇₁	a ₇₂	a ₇₃	a ₇₄	a ₇₅	a ₇₆	a₇₇	a ₇₈
a ₈₁	a ₈₂	a ₈₃	a ₈₄	a ₈₅	a ₈₆	a ₈₇	a₈₈

- 算法描述

- 初始化空棋盘（起始状态）；
- 从第一行开始，直至第一行出现回溯
- 在当前行 r 中查找下一个可以放置王后的位置
- 如果找到了可以摆放的位置
- 放下一个子
- 如果已经是最后一行
- 得到一个解
- 撤掉该子，继续寻找下一个解
- 否则（未到最后一行）
- 准备处理下一行
- 否则（没有找到可以摆放的位置）
- 回溯到上一行，并撤掉该行的棋子



算法6.4: 可以放置一个新皇后吗? (测试 $x(k)$ 是否合理)

Procedure PLACE(k) //相当于 B_k ,考察在放置了前 $k-1$ 个后 x_k 的合理性。

//如果一个皇后能放在第 k 行和 $X(k)$ 列, 则返回true, 否则返回false。
 X 是一个全程数组, 进入此过程时已置入了 k 个值。ABS(r)过程返回 r 的绝对值//

global $X(1:k)$; integer i, k ;

$i \leftarrow 1$

while $i < k$ do //i=k时结束, 前 $i-1$ 个不与第 k 个位置冲突

if $X(i)=X(k)$ or $ABS(X(i)-X(k))=ABS(i-k)$ then

return (false)

end if

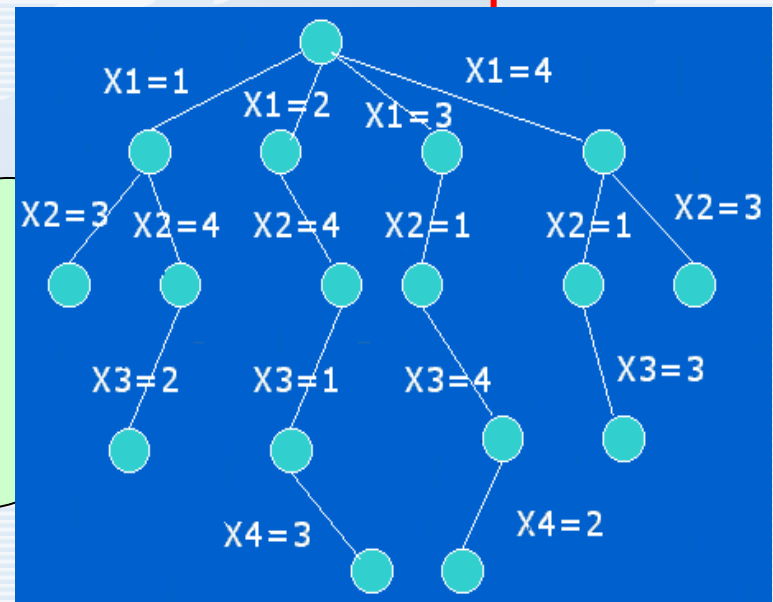
$i \leftarrow i+1$

repeat

return (true)

End PLACE

判断是否有其它的皇后与之在同一列或同一斜对角线上



算法6.5: n-皇后问题的所有解

Procedure NQUEENS(n)

//此过程使用回溯法求出一个 $n \times n$ 棋盘上放置n个皇后，使其不能互相攻击的所有可能位置//

integer k,n,X(1:n)

X(1)←0 ; k←1 // k是当前行; X(k)是当前列 //

while k>0 do // 对所有的行，执行以下语句 //

X(k)←X(k)+1 //移到下一列//

while X(k)≤n and Not PLACE(k) do //此处能放这个皇后吗//

X(k)←X(k)+1 //不能放则转到下一列，取下一位置用//

repeat

if X(k)≤n then //找到一个位置//

if k=n then print (X) //是一个完整的解则打印这个数组//

else k←k+1;X(k)←0 //否则转到下一行扩展，搜索下一皇后//

end if

else k←k-1 //回溯，没有合适位置//

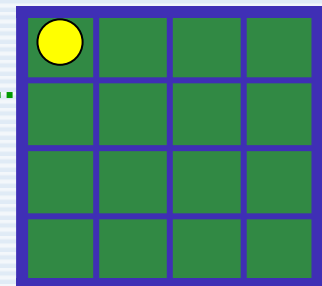
end if

repeat

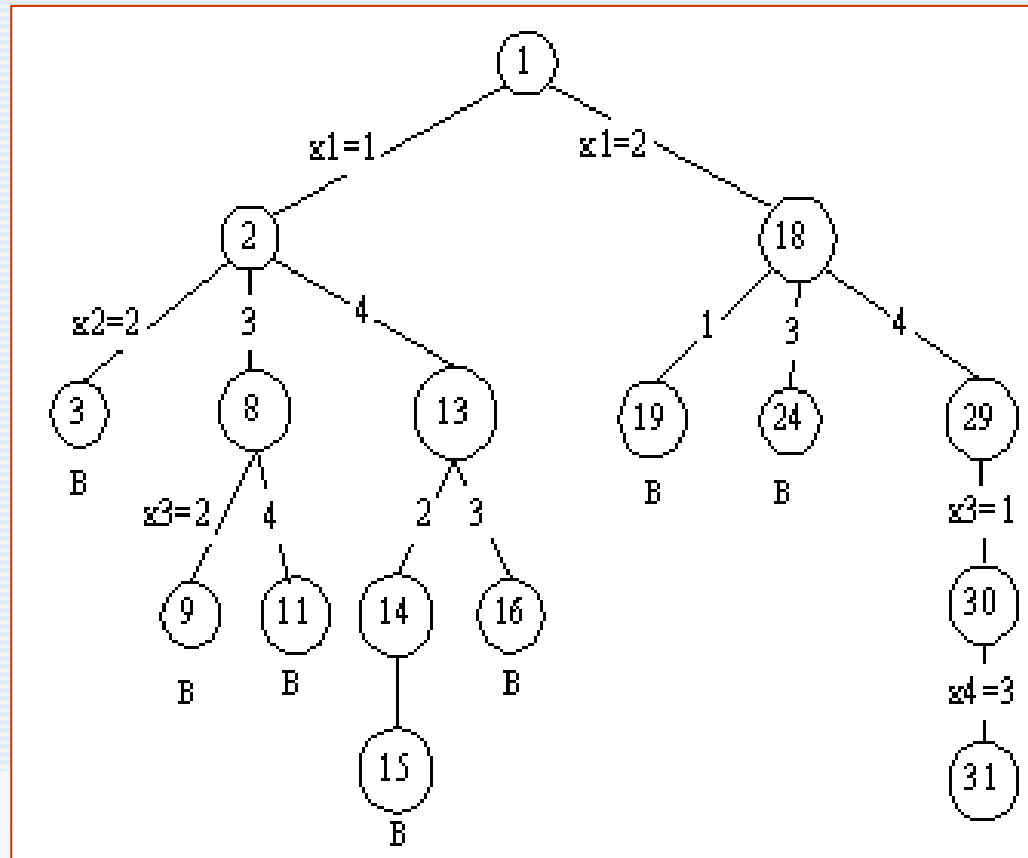
End NQUEENS

2020/10/26

初始



不合理的位置
,x(k)
>n



算法分析：

对于8*8棋盘可能的选择方式是： $\begin{pmatrix} 64 \\ 8 \end{pmatrix}$ ，即 4.4×10^9 个8-元组。而用上述过程至多做8！次检查，即40320个8-元组。

8-皇后状态空间树的结点总数是：

$$1 + \sum_{j=0}^7 (\prod_{i=0}^j (8-i)) = 109601$$